

memory related

1. The method of slab debug locating kernelspace memory leak

The following are methods to assist in locating memory leaks, mainly for memory kmalloc.

1. Open the following two configs in the kernel

```
Symbol: SLUB_DEBUG [=y]
Type : boolean
Prompt: Enable SLUB debugging support
Location:
(3) -> General setup
Defined at init/Kconfig:1752
Depends on: SLUB [=y] && SYSFS [=y]

Symbol: SLUB_DEBUG_ON [=y]
Type : boolean
Prompt: SLUB debugging on by default
Location:
-> Kernel hacking
(4) -> Memory Debugging
Defined at lib/Kconfig.debug:482
Depends on: SLUB [=y] && SLUB_DEBUG [=y] && !KMEMCHECK [=n]
```

2. View meminfo

As shown in the figure below, if SUnreclaim keeps increasing, it can be preliminarily judged that the kmalloc in the kernel has not been released, resulting in a memory leak.

3. slabinfo comparison

Use the backup when booting up `cat /proc/slabinfo`, and use it after a leak occurs `cat/proc/slabinfo`, and compare the two.

116	radix_tree_node	892	1050	280	14	1 : tunables	0	0	0 : slabdata	75	75	0
117	idr_layer_cache	128	143	1248	13	4 : tunables	0	0	0 : slabdata	11	11	0
118	kmalloc-8192	8	9	8448	3	8 : tunables	0	0	0 : slabdata	3	3	0
119	kmalloc-4096	158	165	4352	7	8 : tunables	0	0	0 : slabdata	27	27	0
120	kmalloc-2048	91	98	2304	14	8 : tunables	0	0	0 : slabdata	7	7	0
121	kmalloc-1024	260	264	1280	12	4 : tunables	0	0	0 : slabdata	22	22	0
122	kmalloc-512	409	490	768	10	2 : tunables	0	0	0 : slabdata	49	49	0
123	kmalloc-256	356	464	512	8	1 : tunables	0	0	0 : slabdata	58	58	0
124	kmalloc-192	100	117	448	9	1 : tunables	0	0	0 : slabdata	13	13	0
125	kmalloc-128	1167	1180	384	10	1 : tunables	0	0	0 : slabdata	118	118	0
126	kmalloc-64	14587	14712	320	12	1 : tunables	0	0	0 : slabdata	1226	1226	0
127	kmem_cache_node	119	120	320	12	1 : tunables	0	0	0 : slabdata	10	10	0
128	kmem_cache	119	120	384	10	1 : tunables	0	0	0 : slabdata	12	12	0

4. Analyze the data above

In the figure, the number of kmalloc-8192 has increased a lot.

`/sys/kernel/slab/kmalloc-8192/alloc_calls` Check the symbol that calls kmalloc-8192 and the number of times it calls, and locate the leaked function.

2. kmemleak--Kernel space memory leak analysis tool

2.1. Memory application for kernel mode

There are two main ways to apply for kernel mode memory in the SDK:

1. vmalloc

分配大块内存，走buddy system；通过 `cat /proc/vmallocinfo` 可以统计；

注：`Cat vmallocinfo`，如果有buffer块数一直在增加，则是内存泄露。

2. kmalloc/kmem_cache_create

分配小于pagesize，走slab机制；通过 `cat /proc/meminfo` 里的slab字段可以统计。

注：如果怀疑有kernel mode内存泄露，运行应用过程中每间隔一段时间 `cat /proc/meminfo` 留意slab。如果一直在增加，大概率有内存泄露的可能。具体模块的函数泄露可以用kmemleak debug。

2.2. 检查内存泄漏的方法 -- Kernel Space

Linux kernel 2.6.31后的版本，提供了 KMEMLEAK 的选项，可以拿来测试 kernel modules是否有 memory leakage，用法整理如下。

1. 修改 `.config` 设定 KMEMLEAK，重新编译 kernel，重烧 image。

摘录 `.config` 如下：

```
#
# Memory Debugging
#
CONFIG_HAVE_DEBUG_KMEMLEAK=y
CONFIG_DEBUG_KMEMLEAK=y
CONFIG_DEBUG_KMEMLEAK_EARLY_LOG_SIZE=4096
```

2. 确认板子上的 kernel 已经提供 kmemleak的功能

```
# mount -t debugfs nodev /sys/kernel/debug/

# cat /sys/kernel/debug/kmemleak
```

注：若存在 kmemleak，表示 kernel 已经支援 kmemleak

3. 用法举例

```
[/system/vendor]## mount -t debugfs nodev /sys/kernel/debug/
[/system/vendor]## echo clear > /sys/kernel/debug/kmemleak
[/system/vendor]## insmod ./kmleak.ko
pointer to the allocated memory: c2420040
[/system/vendor]## echo scan > /sys/kernel/debug/kmemleak
[/system/vendor]## echo scan > /sys/kernel/debug/kmemleak
kmemleak: 1 new suspected memory leaks (see /sys/kernel/debug/kmemleak)
[/system/vendor]## cat /sys/kernel/debug/kmemleak
unreferenced object 0xc2420040 (size 64):
  comm "insmod", pid 130, jiffies 100113 (age 21.670s)
  hex dump (first 32 bytes):
    6d 65 6d 6f cat (131) used greatest stack depth: 4944 bytes left
    72 79 20 6c 65 61 6b 20 74 65 73 74 memory leak test
    00 61 6b 5d 00 00 00 00 00 00 00 00 80 bd 05 c0 .ak].....
  backtrace:
    [] leak_func+0x50/0xcc [kmleak]
    [] kmleak_init+0x8/0x10 [kmleak]
    [] do_one_initcall+0x188/0x1c4
    [] load_module+0x14a0/0x1a44
    [] Sys_init_module+0xc0/0xd8
    [] ret_fast_syscall+0x0/0x38
    [] 0xffffffff
[/system/vendor]##
```

cid:image003.png\\@01D2DB8B.2E89DE10

注：若沒有手动执行 scan，系统每10分钟自动扫描一次 memory leakage.

4. 实际测试 Driver memory leakage 的方法

```
# mount -t debugfs nodev /sys/kernel/debug/
# echo clear > /sys/kernel/debug/kmemleak
# run_your_driver_test_here
```

```
# echo scan > /sys/kernel/debug/kmemleak
# cat /sys/kernel/debug/kmemleak
```

3. dmalloc--查usr space内存泄露

3.1. 内存泄漏排查

程序运行过程meminfo的Memfree不断减少，`echo 3`
`>/proc/sys/vm/drop_cache`也不返回内存，则可能存在内存泄露。

排除kernel space内存泄露后，隔段时间再观察 `cat /proc/PID/smap` 每个so的堆，可以找到内存泄漏的so。

准确定位内存泄露需使用dmalloc工具。

3.2. 交叉编译

1. <http://dmalloc.com/> [<http://dmalloc.com/>]可下载dmalloc-5.5.2.gz
解压，修改configure文件，`ac_cv_page_size=0` 改为`ac_cv_page_size=12`
 2. `./configure --prefix=/home/cbm/workplace/dmalloc/output/ --exec-prefix=/home/cbm/workplace/dmalloc/output/ CC=arm-buildroot-linux-uclibcgnueabi-gcc --host=arm-linux CXX=arm-buildroot-linux-uclibcgnueabi-g++ AR=arm-buildroot-linux-uclibcgnueabi-ar LD=arm-buildroot-linux-uclibcgnueabi-ld --enable-threads --enable-shlib`
 3. auto脚本对交叉编译支持不够完善，生成的Makefile有少数命令需替换
如 `ld-G · ar cr` 需改为 `$(LD) -G` 和 `$(AR) cr`
 4. `make CC=arm-buildroot-linux-uclibcgnueabi-gcc CXX=arm-buildroot-linux-uclibcgnueabi-g++ AR=arm-buildroot-linux-uclibcgnueabi-ar LD=arm-buildroot-linux-uclibcgnueabi-ld`
 5. `make install`，即可在prefix目录生成bin include和lib。
-

3.3. 环境变量设置

dmalloc工具进行环境变量设置，如：`dmalloc -l /tmp/dmalloc.log all`，即输出所有信息并存入 `dmalloc.log` 文件；

```
# dmalloc -l /tmp/dm.log all
DMALLOC_OPTIONS=debug=0xcf4ed2b,log=/tmp/dm.log `
export DMALLOC_OPTIONS `
```

3.4. 库的应用

1. 在主代码中添加 `#include <dmalloc.h>`
 2. 更改Makefile，添加 `CFLAGS += -Iinclude` `LDFLAGS += -L$(TOP)/dmalloc`
`-ldmalloc`
 3. 重新build需要detect的应用
-

3.5. Leak检测

1. 通过dmalloc所生产的变量DMALLOC_OPTIONS，手动或自动export到系统中
 2. 执行需detect的应用
 3. 查看变量DMALLOC_OPTIONS中所定义的导出文件内容
 4. 退出需detect的应用
 5. 查看变量DMALLOC_OPTIONS中所定义的导出文件内容
 6. 上述过程所查看的导出文件存在 `not freed:` 字串，则为leak点
-

4. 程序运行内存占用

以公板运行mixer为例，详细说明内存如何分配。

4.1. 实验设备

335+imx307 sensor

4.2. 实验场景

(典型案例) mixer路4路流：JPEG + 主码流 + 子码流 + MD算法

4.3. 实验基础

SDK里kernel space 下ko有两种内存申请方式：

- vmalloc

分配大块内存，走budysystem；通过 `cat/proc/vmallocinfo` 可以统计；

- kmalloc/kmem_cache_create

分配小于pagesize，走slab机制；通过 `cat/proc/meminfo` 里的slab字段可以统计。

内核中常见的还有如`alloc_pages/__get_free_page`，只能通过free统计，但目前看起来只有emmc会调用。

- loremap

I/O内存资源的物理地址映射到核心虚地址空间，不占用物理空间；

- PageTable

用于将内存的虚拟地址翻译成物理地址，`/proc/meminfo`中的PageTables统计了Page Table所占用的内存大小。

- KernelStack

每一个用户线程都会分配一个kernel stack（内核栈），它是kernel消耗的内存。统计值是`/proc/meminfo`的KernelStack。

Userspace中常见malloc，小于128K走c标准库的brk，大于128K走mmap，但对于内核来说，一旦发生缺页中断，都是走pageallocator。可以通过procmem工具统计实际占用的物理空间（实际上就是把`cat/proc/PID/smap`里的RSS/PSS之和）。

4.4. 实验步骤

1. 开机加载完ko后，drop cache再看 `cat proc/meminfo` 和 `cat /proc/vmallocinfo`
 2. 运行mixer后，drop cache再看 `cat proc/meminfo`、`cat /proc/vmallocinfo`、`cat/proc/'PID'/status` 和 `./procmem PID`
 3. 对比工具分析运行前后的vmalloc增加部分
-

4.5. 总结

free减少 = $16836 - 8888 = 7948\text{K}$

Vmalloc增加：547page = 2188K

lab增加：6328 - 4792 = 1536K

PageTable+KernelStack增加：536+116 - (352+52) = 248K

应用代码段+ so 代码段 + 栈 + ro段+堆 = 3732K (Pss)

$2188 + 1536 + 248 + 3732 = 7704\text{K}$ ，Linux kernel并没有统计所有的内存分配，kernel动态分配的内存中有一部分没有计入 `/proc/meminfo`。我们只能大概的统计出来mixer的内存占用。

5. procmem procrank--usr space内存分析工具

源码下载：https://github.com/sunao2002002/mem_proc.git

[https://github.com/sunao2002002/mem_proc.git]

1. 解压后用下文内容替换 `CMakeLists.txt`。

```
SET(CMAKE_C_COMPILER "arm-buildroot-linux-uclibcgnueabi-hf-gcc")
SET(CMAKE_CXX_COMPILER "arm-buildroot-linux-uclibcgnueabi-hf-g++")
cmake_minimum_required(VERSION 2.8)
```

```
PROJECT (mem_proc)
file(GLOB SOURCES "${PROJECT_SOURCE_DIR}/libpagemap/*.c")
add_definitions (
-D_LARGEFILE64_SOURCE
)
include_directories (${PROJECT_SOURCE_DIR}/libpagemap/include)
add_library(pagemap ${SOURCES})

add_executable(procmem
${PROJECT_SOURCE_DIR}/procmem/procmem.c)
target_link_libraries(procmem pagemap)
add_executable(procrank
${PROJECT_SOURCE_DIR}/procrank/procrank.c)
target_link_libraries(procrank pagemap)
```

2. cd mem_proc
3. mkdir out
4. cd out
5. cmake ..
6. make 即可生成procmem和procrank

VSS : Virtual Set Size , 虚拟内存耗用内存 , 包括共享库的内存

RSS : Resident Set Size , 实际使用物理内存 , 包括共享库

PSS : Proportional Set Size , 实际使用的物理内存 , 共享库按比例分配

USS : Unique Set Size, the physical memory exclusive to the process, does not count the shared library, it can also be understood as killing the process to release